

Fuzzy Private Set Union avec Chiffrement Homomorphe de Classe $\mathcal{CL}$

Intégration de BICYCL dans volePSI – Comparaison et Évaluation

Yasser NAMATY

Laboratoire Jean Kuntzmann

Encadrants : Aude Maignan / Jean-Guillaume Dumas

2026

Plan de la présentation

- 1 Contexte et problématique
- 2 Implémentation
- 3 Résultats expérimentaux
- 4 Analyse et perspectives

Cadre

- Projet mené au **Laboratoire Jean Kuntzmann (LJK)**
- Encadrement : Aude Maignan, Jean-Guillaume Dumas
- Domaine : cryptographie appliquée, calcul multipartite sécurisé (MPC)

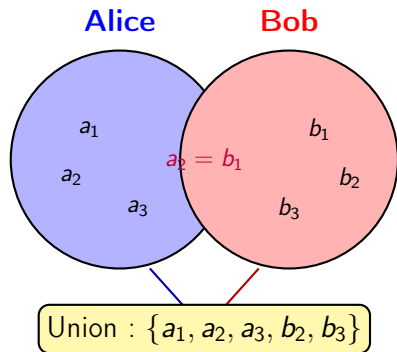
Objectif du projet

- Intégrer la bibliothèque **BICYCL** (chiffrement homomorphe de classe $\mathcal{CL}$) dans un protocole de **Fuzzy Private Set Union** construit sur **volePSI**
- Évaluer et comparer les performances obtenues face au chiffrement de **Paillier**, classiquement utilisé dans ce contexte

Private Set Union (PSU) : définition classique

Problème

Deux parties, chacune détenant un ensemble d'éléments privés, souhaitent calculer l'union de leurs ensembles sans révéler d'informations supplémentaires.



- Applications :
 - Données médicales
 - Détection de menaces
 - Publicité ciblée
- **Objectif** : Calculer l'union sans révéler les éléments
- Ici, a_2 et b_1 sont fusionnés car **strictement égaux**

De la PSU classique à la *Fuzzy* PSU

Limite de la PSU classique

La PSU classique ne fusionne que des éléments **strictement identiques**. Or, beaucoup de données réelles sont **bruitées** : deux mesures du même objet ne sont presque jamais bit-à-bit identiques.

Exemples de données “proches mais pas égales”

- **Biométrie** : deux scans du même doigt donnent deux empreintes légèrement différentes
- **Géolocalisation** : deux relevés GPS d'un même lieu diffèrent de quelques mètres
- **Génomique** : deux séquences ADN homologues comportent quelques mutations
- **Chaînes de caractères** : fautes de frappe, variantes orthographiques (noms, adresses)

Idée de la *Fuzzy* PSU

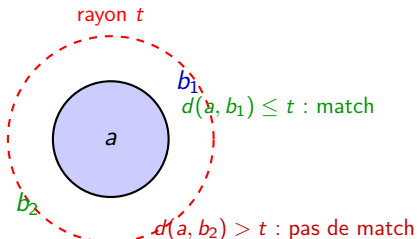
Remplacer l'égalité stricte $a = b$ par un critère de **proximité** : deux éléments sont fusionnés dès qu'ils sont “assez proches”, selon une distance choisie et un seuil de tolérance.

Définition

Soit (X, d) un espace métrique et $t \geq 0$ un seuil de tolérance. Pour $x \in A$ et $y \in B$, on dit que x et y **matchent** si

$$d(x, y) \leq t.$$

La *Fuzzy* PSU calcule alors une union où les paires (x, y) qui matchent sont fusionnées en une seule entrée, au lieu d'apparaître comme deux éléments distincts.



- Distance d typique :
 - Hamming (bits, ADN)
 - Euclidienne (coordonnées GPS)
 - Édition (chaînes de caractères)
- Le choix de t traduit un compromis **précision / robustesse au bruit**

PSU classique vs *Fuzzy* PSU

Critère	PSU classique	<i>Fuzzy</i> PSU
Relation de fusion	Égalité stricte ($=$)	Proximité ($d(x, y) \leq t$)
Structure sous-jacente	Hachage / OKVS	OKVS + distance (Hamming, LSH, buckets)
Complexité naïve	$\mathcal{O}(n + m)$	$\mathcal{O}(n \times m)$ sans optimisation
Complexité optimisée	$\mathcal{O}(n + m)$	quasi-linéaire (buckets / LSH)
Données visées	Identifiants exacts (emails, IDs)	Données bruitées (biométrie, géoloc, ADN)

À retenir

La *Fuzzy* PSU généralise la PSU classique : cette dernière correspond au cas particulier $t = 0$ (égalité stricte). Le protocole étudié dans ce projet (FPSU, basé sur volePSI) s'inscrit dans ce cadre *fuzzy*.

État de l'art et positionnement

PSI / PSU classiques

- Protocoles à base d'**Oblivious Transfer** (OT) et de fonctions pseudo-aléatoires obliées (OPRF)
- Protocoles récents à base d'**OKVS** et de corrélations **VOLE** (ex. volePSI), offrant de meilleures performances en communication et en temps

Fuzzy PSI / PSU

- Domaine moins mature que le cas classique : la comparaison “par distance” complique le passage à l'échelle
- Approches usuelles : découpage en *buckets*, hachage sensible à la localité (LSH), ou combinaison avec un chiffrement homomorphe pour vérifier la proximité sans la révéler

Positionnement du projet

Ce travail se concentre sur un maillon spécifique de la chaîne *Fuzzy* PSU : remplacer le

Le chiffrement homomorphe : principe

Chiffrement classique

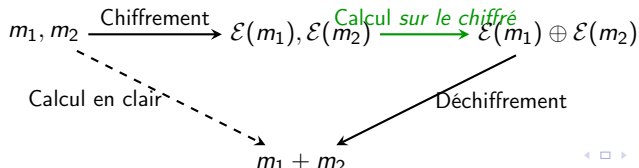
Avec un chiffrement classique (AES, RSA...), le chiffré c est “opaque” : dès qu’une donnée est chiffrée, on ne peut plus rien calculer dessus sans la déchiffrer d’abord.

Chiffrement homomorphe

Un chiffrement est **homomorphe** s’il existe une opération $\oplus$ sur les chiffrés telle que

$$\mathcal{D}(\mathcal{E}(m_1) \oplus \mathcal{E}(m_2)) = m_1 + m_2,$$

sans jamais déchiffrer $\mathcal{E}(m_1)$ ni $\mathcal{E}(m_2)$, et sans connaître la clé privée.



À quoi sert le chiffrement homomorphe ?

Idée générale

Permettre à une machine (un serveur, un tiers non fiable) de **calculer sur des données privées** sans jamais les voir en clair.

Exemples classiques

- **Cloud computing** : déléguer un calcul à un serveur sans lui révéler les données
- **Vote électronique** : additionner des bulletins chiffrés sans les déchiffrer individuellement
- **Statistiques médicales** : agréger des données de patients sans exposer chaque dossier
- **Calcul multipartite (MPC)** : combiner des parts secrètes détenues par plusieurs parties

Dans ce projet

Le protocole FPSU utilise un chiffrement **additif** homomorphe (pas besoin de multiplication) pour vérifier une égalité *sans révéler les valeurs comparées* – voir la diapositive suivante.

Le chiffrement homomorphe dans le protocole FPSU

Le problème à résoudre

Chaque serveur détient un *partage* s_1 ou s_2 d'une valeur secrète, avec $s_1 + s_2 = 0$ si et seulement s'il y a un "match". Aucun des deux serveurs ne doit apprendre s_1 ou s_2 individuellement.

Ce que permet le chiffrement homomorphe additif

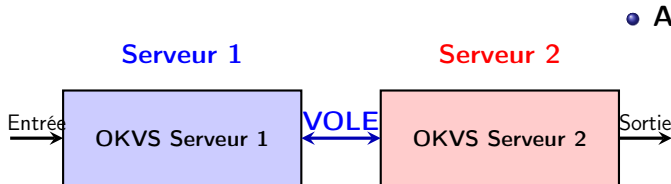
- 1 Chaque serveur chiffre son partage : $\mathcal{E}(s_1)$, $\mathcal{E}(s_2)$
- 2 Les chiffrés circulent et sont **additionnés directement**, sans déchiffrement intermédiaire :
 $\mathcal{E}(s_1) \oplus \mathcal{E}(s_2) = \mathcal{E}(s_1 + s_2)$
- 3 Seul le résultat final $\mathcal{D}(\mathcal{E}(s_1 + s_2))$ est révélé
- 4 Si ce résultat vaut 0 $\Rightarrow$ match ; sinon $\Rightarrow$ pas de match

Pourquoi c'est essentiel

Sans homomorphie, il faudrait déchiffrer s_1 et s_2 séparément pour tester l'égalité – ce qui

FPSU (*Fuzzy Private Set Union*)

- Basé sur **volePSI** (VOLE-based PSI)
- Utilise des corrélations VOLE (Vector Oblivious Linear Evaluation)
- Encodage des ensembles via **OKVS** (Paxos)
- Comparaison *floue* des éléments (au sens de la diapositive précédente), et non une simple égalité



• Avantages :

- Communication faible
- Temps quasi-linéaire
- Sécurité prouvée

Chiffrement Homomorphe de Classe $\mathcal{CL}$

Définition

- Groupe de classes d'idéaux de corps quadratiques imaginaires
- Schéma $\mathcal{CL}$ -**HSMqk** (Castagnos-Laguillaumie)
- Chiffrement **additif** homomorphe

Propriétés mathématiques

$$\text{Cl}(\Delta_K) = \frac{\text{Idéaux fractionnaires}}{\text{Idéaux principaux}}$$

$$\mathcal{E}(m_1 + m_2) = \mathcal{E}(m_1) \oplus \mathcal{E}(m_2)$$

Caractéristiques

- Sécurité 128 bits : $q = 256$ bits
- Chiffrés : 4663 bits
- Add homomorphe : 11 ms
- Clés compactes

Comment fonctionne BICYCL ? – Le groupe de classes

Représentation : les formes quadratiques

Un élément du groupe de classes $\text{Cl}(\Delta_K)$ est représenté par une **forme quadratique binaire** (a, b, c) de discriminant

$$\Delta = b^2 - 4ac.$$

L’“addition” du groupe correspond à une opération de **composition** entre deux formes.

Pourquoi ce groupe ?

- Son **ordre est inconnu** (difficile à calculer), comme pour RSA
- **Mais sans paramètre secret** : pas de module $n = pq$ à générer et à protéger (contrairement à Paillier)
- $\Rightarrow$ pas de “trusted setup”

Composition efficace : NUCOMP

- L’algorithme **NUCOMP** (et **NUDUPL** pour le doublement) calcule la composition de deux formes rapidement
- Remplace l’algorithme historique de Gauss, bien plus lent
- C’est l’opération de base répétée dans

Comment fonctionne BICYCL ? – Chiffrer et déchiffrer

Deux sous-groupes emboîtés

- Un sous-groupe d'ordre **connu** et premier q : sert à **encoder le message** m (via un générateur f de ce sous-groupe)
- Le reste du groupe, d'ordre **inconnu** : sert à **masquer** le message (aléa de chiffrement)

Vue d'ensemble du schéma

- 1 **KeyGen** : tire une clé secrète x , publie $pk = g^x$ dans le groupe de classes
- 2 **Encrypt**(m) : tire un aléa r , calcule $c_1 = g^r$ et $c_2 = pk^r \cdot f^m$ (composition de formes, via NUCOMP)
- 3 **Decrypt** : retire le masque avec x , puis retrouve m car il vit dans le **petit** sous-groupe d'ordre q (calcul de logarithme discret facile car q est connu et petit)
- 4 **Add** : composer directement les formes des deux chiffrés $\Rightarrow$ chiffré de la somme

BICYCL

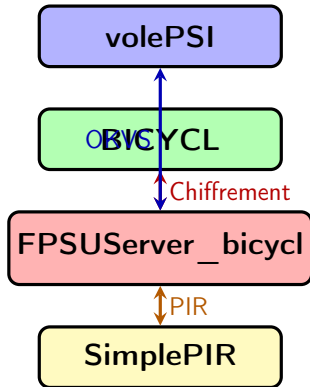
- Bibliothèque C++ du LIRMM
- Implémente $\mathcal{CL}$ -HSMqk
- Interface moderne (C++20)

Caractéristiques

- Opérations sur formes quadratiques (NUCOMP)
- Compatible avec volePSI
- Paramètres flexibles

Pourquoi BICYCL ?

- Chiffrés plus compacts que Paillier
- Chiffrement plus rapide
- Meilleure intégration C++



- OKVS (Paxos)
- VOLE
- OT
- $\mathcal{CL}$ -HSMqk
- Requêtes PIR

Définition

```
constexpr size_t kMaxValueBytes = 4096;  
constexpr size_t kMaxValueBlocks = kMaxValueBytes / sizeof(block);  
using ValueType = std::array<block, kMaxValueBlocks>;
```

Opérateurs requis

- `operator^`
- `operator^=`
- `operator&`

Sérialisation

- Chiffré $\rightarrow$ ValueType
- ValueType $\rightarrow$ Chiffré
- Taille fixe : 4096 octets

Encodage compact

- Chaque QFI encode : (a, b, c)
- Marqueur de signe + longueur
- Taille totale : variable

```
std::vector<unsigned char> encode_mpz(const BICYCL::Mpz& value) {  
    std::vector<unsigned char> abs_bytes = value;  
    std::vector<unsigned char> out;  
    out.push_back(marker);  
    append_u32(out, abs_bytes.size());  
    out.insert(out.end(), abs_bytes.begin(), abs_bytes.end());  
    return out;  
}
```

Format

Octet 0	Octets 1-4	Octets 5-N
---------	------------	------------

Modifications clés

- 1 Définition de `ValueType` adaptée
- 2 Sérialisation BICYCL $\leftrightarrow$ OKVS
- 3 Reconstruction homomorphe des partages

Flux du protocole

- 1 Chiffrement des partages : $\mathcal{E}(s_1), \mathcal{E}(s_2)$ avec $s_1 + s_2 = 0$
- 2 Encodage OKVS avec les chiffrés
- 3 Décodage et addition homomorphe
- 4 Vérification : $\mathcal{D}(\mathcal{E}(s_1) + \mathcal{E}(s_2)) = 0$

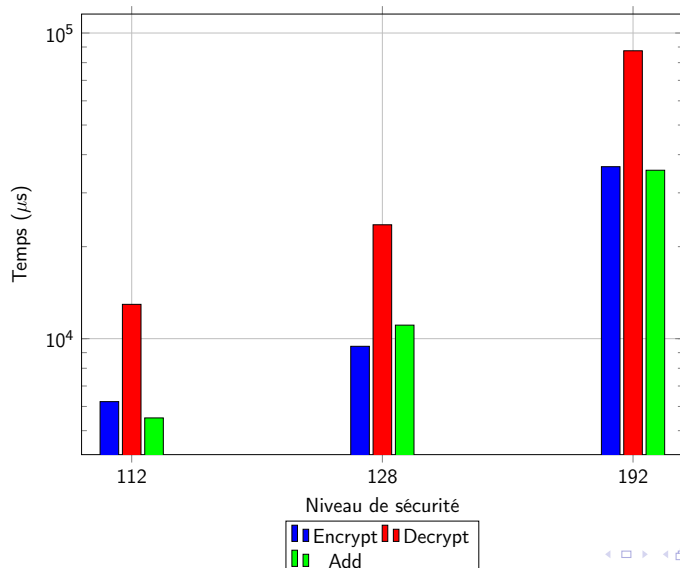
Benchmark BICYCL vs Paillier (SecLevel 128)

	BICYCL	PHPE	SHPE	Gain
CT (bits)	4 663	6 144	6 144	+24%
KeyGen (ms)	24.45	183.76	1353.04	×7–55
Encrypt (ms)	9.44	84.75	163.63	×9–17
Decrypt (ms)	23.58	22.36	3.74	×6
Add (μ s)	11 073	70	70	×158

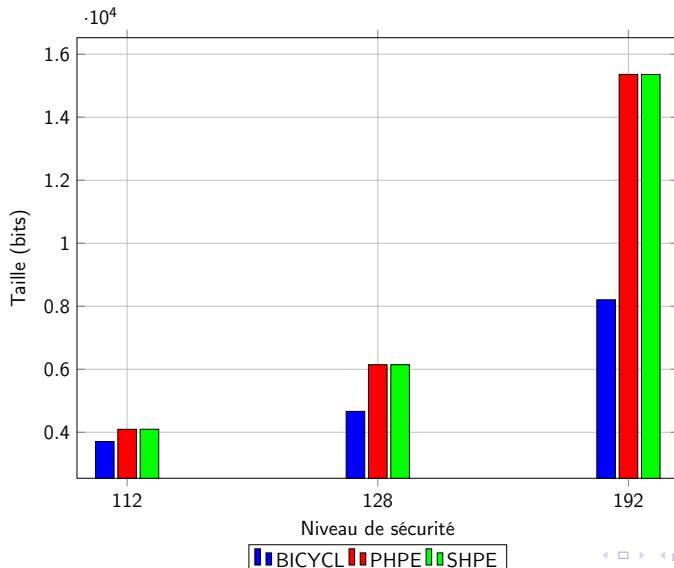
Observations

- BICYCL : Chiffrés plus compacts, chiffrement plus rapide
- Paillier : Addition homomorphe plus rapide
- **Compromis favorable pour FPSU**

Temps d'exécution BICYCL



Taille des chiffrés



Variante	Setup (ms)	Décodage (ms)	Total (ms)
FPSUServer (base)	0.13	0.004	0.134
FPSUServer_bicycl	31.20	2.50	33.70
Facteur	×240	×625	×251

Analyse

- Surcoût : facteur ≈ 250
- Principalement dû au chiffrement (16 chiffrements $\times 9.44$ ms)
- **Acceptable** pour des ensembles de taille modérée ($n \leq 100$)

Résultats de décodage

Test avec $n = 8$, $m = 5$

- Indices 0 et 1 : Clés synchronisées → **Succès**
- Indice 2 : Clés désynchronisées → **Échec**
- Indices 3 et 4 : Clés aléatoires → **Échec**

Sortie console

```
decode succeeded: The server already  
has the number 0  
decode succeeded: The server already  
has the number 1  
decode failed: The server does not  
have the number value 2  
decode failed: The server does not  
have the number value 3
```

Interprétation

- Décodage OKVS correct
- Reconstruction homomorphe valide
- Détection des éléments hors ensemble

Fonctionnalités

- Requêtes PIR sur les OKVS stockés
- Cache les indices demandés
- Deux serveurs indépendants

```
// Setup des serveurs PIR
PIRServerSetup(1, storage[0].size());
PIRServerSetup(2, storage[1].size());

// Requête client pour l'index 3
PIRClientQuery(3);
PIRServerAnswer(1);
PIRServerAnswer(2);
PIRClientExtract();

// Décodage classique
paxos.decode<ValueType>(clientSets[0], decoded[0], storage[0]);
paxos.decode<ValueType>(clientSets[1], decoded[1], storage[1]);
```

Comparaison BICYCL vs Paillier

Table – SecLevel 128, $k = 256$ bits

Critère	BICYCL	Paillier	Vainqueur
CT bits	4 663	6 144	BICYCL
KeyGen (ms)	24.45	183.76	BICYCL
Encrypt (ms)	9.44	84.75	BICYCL
Decrypt (ms)	23.58	22.36	Paillier
Add (μs)	11 073	70	Paillier

Conclusion

BICYCL est **compétitif** pour FPSU :

- Chiffrement plus rapide
- Clés plus compactes
- Meilleure intégration C++

Complexité

- Chiffrement : $\mathcal{O}(n)$
- Décodage : $\mathcal{O}(m)$
- Reconstruction : $\mathcal{O}(m)$
- **Linéaire en n et m**

Estimations (n éléments)

n	Temps estimé (ms)
8	33.7
100	≈ 420
1000	≈ 4200

Scalabilité

- Linéaire en la taille des ensembles
- Parallélisation possible
- Adapté aux applications réelles

Limites actuelles

- Taille fixe de `ValueType` (4096 octets)
- Surcharge de sérialisation
- Tests limités à $n = 8$

Améliorations possibles

- 1 Compression des QFI (plus compacte)
- 2 Batch encryption / decryption
- 3 Utilisation de $k > 1$ pour plus de sécurité
- 4 Optimisation du `operator^` sur `ValueType`

Travaux futurs

- **Extension** à de plus grands ensembles ($n \geq 10^3$)
- **Optimisation** de la sérialisation
- **Parallélisation** des opérations
- **Comparaison** avec d'autres schémas homomorphes
- **Formalisation** de la sécurité

Applications

- **Médical** : Partage sécurisé de données patients
- **Réseau** : Détection collaborative de menaces
- **Finance** : KYC inter-bancaire
- **IA** : Apprentissage fédéré

Contributions

- 1 Implémentation de FPSUServer_bicycl
- 2 Intégration de BICYCL dans volePSI
- 3 Campagne de benchmarks BICYCL vs Paillier
- 4 Évaluation du protocole *Fuzzy* PSU avec chiffrement homomorphe

Résultats clés

- BICYCL : +24% de compacité, $\times 9$ chiffrement
- Surcoût FPSU : facteur 250 (acceptable)
- **Compromis favorable** pour les applications modérées

Je remercie chaleureusement

Aude Maignan et **Jean-Guillaume Dumas**

pour leur encadrement tout au long de ce projet,

ainsi que l'ensemble du **Laboratoire Jean Kuntzmann**
pour son accueil et les échanges scientifiques.

- G. Castagnos, F. Laguillaumie – *Linearly Homomorphic Encryption from DDH*, CT-RSA 2015 (schéma $\mathcal{CL}$ -HSM)
- G. Castagnos, F. Laguillaumie, I. Tucker *et al.* – bibliothèque **BICYCL** (LIRMM)
- P. Rindal *et al.* – **volePSI**, PSI/PSU basé sur OKVS et corrélations VOLE
- Littérature sur la *Fuzzy* PSI/PSU : approches par *buckets*, distance de Hamming, hachage sensible à la localité (LSH)

Remarque

Cette diapositive liste les familles de travaux mobilisés ; les références précises (auteurs, année, venue) sont à vérifier et compléter à partir de la bibliographie du mémoire avant la soutenance.

Merci pour votre attention

Merci pour votre attention !

Questions ?

Code source : sérialisation I

```
void ciphertext_to_valuetype(const BICYCL::CL_HSMqk::CipherText& ct, ValueType& out)
{
    std::vector<unsigned char> payload;
    std::vector<unsigned char> c1_bytes = encode_qfi(ct.c1());
    std::vector<unsigned char> c2_bytes = encode_qfi(ct.c2());
    payload.insert(payload.end(), c1_bytes.begin(), c1_bytes.end());
    payload.insert(payload.end(), c2_bytes.begin(), c2_bytes.end());

    std::fill(out.begin(), out.end(), ZeroBlock);
    std::uint32_t payload_len = static_cast<std::uint32_t>(payload.size());
    const size_t header_size = sizeof(payload_len);
    if (header_size + payload.size() > kMaxValueBytes) {
        throw std::runtime_error("Serialized ciphertext is too large");
    }

    std::memcpy(out.data(), &payload_len, header_size);
    std::memcpy(reinterpret_cast<unsigned char*>(out.data()) + header_size,
                payload.data(), payload.size());
}
```

Code source : reconstruction homomorphe I

```
for (u64 i = 0; i < m; ++i)
{
    bool found = false;
    try {
        BICYCL::CL_HSMqk::CipherText c1 =
            valuetype_to_ciphertext(crypto_system, decoded1[i]);
        BICYCL::CL_HSMqk::CipherText c2 =
            valuetype_to_ciphertext(crypto_system, decoded2[i]);

        BICYCL::CL_HSMqk::CipherText c_sum =
            crypto_system.add_ciphertexts(public_key, c1, c2, randgen);

        BICYCL::CL_HSMqk::ClearText decrypted_sum =
            crypto_system.decrypt(secret_key, c_sum);

        BICYCL::Mpz zero(0u1);
        found = (decrypted_sum == zero);
    } catch (const std::exception&) {
        found = false;
    }
}
```

Code source : reconstruction homomorphe II

```
if (found)
    std::cout << "decode succeeded: The server already has the number " << i <<
        "\n";
else
    std::cout << "decode failed: The server does not have the number value " <<
        i << "\n";
}
```

Paramètres

- q : module (256 bits pour SecLevel 128)
- k : dimension du sous-groupe
- Δ_K : discriminant du corps quadratique

Opérations

- **KeyGen** : Génère (sk, pk)
- **Encrypt** : $\mathcal{E}(m) \in \text{Cl}(\Delta_K)^2$
- **Decrypt** : $\mathcal{D}(c) = m$
- **Add** : $\mathcal{E}(m_1) \oplus \mathcal{E}(m_2) = \mathcal{E}(m_1 + m_2)$